

YUTA Workflow v3 — Hướng dẫn vận hành

Official Operating Guide - Alternative Layout

APPROVED source / Presentation B for comparison

Status: APPROVED

Visibility: Engineering

Owner: YUTA product and engineering

Last updated: 2026-09-03

1. Workflow này là gì

YUTA Workflow v3 là mô hình vận hành một thay đổi từ ý tưởng đến khi repository được đóng lại có kiểm soát. Mục tiêu là giúp Product, reviewer và engineering cùng nhìn một thay đổi qua các lớp bằng chứng khác nhau, thay vì để một bản mô tả, một lệnh kiểm tra hoặc một trạng thái kỹ thuật tự thay thế quyết định của con người. Nguyên tắc xuyên suốt là:

- Understand broadly
 - decide narrowly
 - implement narrowly
 - verify independently
 - consolidate knowledge

Workflow tách riêng sáu việc:

1. **Product thinking** xác định vấn đề, phạm vi và authority (nguồn có thẩm quyền cho loại câu hỏi đang xét).
 2. **Specs** mô tả hành vi quan sát được cần có.
 3. **Implementation** hiện thực phạm vi đã duyệt trong code, database hoặc UI.
 4. **VERIFY** chứng minh implementation khớp Specs/Design và quy tắc kỹ thuật.
 5. **QA** chứng minh hành vi thực tế trong ngữ cảnh người dùng/runtime phù hợp.
 6. **Normative sync và Knowledge Consolidation** lần lượt quảng bá yêu cầu đã duyệt vào main specs và đối soát lại knowledge hiện hành.
- Release, deploy và Production Readiness không nằm trong định nghĩa repository DONE; chúng thuộc một lane vận hành riêng.

Navigation / sections 1-18

- | | |
|--|----------------------------------|
| 1. Workflow này là gì / 1 | 10. Gate 3 readiness / 8 |
| 2. Vai trò và trách nhiệm / 2 | 11. Normative specs và sync / 8 |
| 3. Tổng quan end-to-end / 3 | 12. \$yuta-run-change / 9 |
| 4. Routing cross-module trước một change / 3 | 13. \$yuta-finish-change / 9 |
| 5. Hướng dẫn vận hành từng bước / 4-5-6 | 14. Knowledge Consolidation / 10 |
| 6. Analysis gate / 7 | 15. No-spec path / 10 |
| 7. Review gates và integrity / 7 | 16. Release / Deploy lane / 10 |
| 8. Tasks, phased implementation và Technical Implementation Contract / 7 | 17. Ví dụ hằng ngày / 10-11 |
| 9. VERIFY khác QA / 8 | 18. Quick reference / 11 |

2. Vai trò và trách nhiệm

Vai trò / lớp authority	Trách nhiệm chính	Không được tự suy ra
Page Chat	Giữ Product context của một page; thực hiện Cross-Module Impact Check; review các gate page-local.	Không khởi tạo change page-local khi kết quả là CROSS_MODULE hoặc UNCERTAIN.
YUTA Control Tower	Điều phối cross-page/module/runtime; làm rõ owner, authority, shared contract và review routing.	Không thay thế chiều sâu Product context của Page Chat hoặc tự giải quyết blocker bằng assumption.
Codex	Đọc authority, tạo/tiếp tục artifact, implement, kiểm tra, tập hợp evidence và dừng đúng gate.	Không phải Product approver; không tự tạo approval, permission, owner hoặc durable boundary.
OpenSpec	Giữ proposal, analysis, delta specs, design và tasks của một change có cấu trúc.	Change trong openspec/changes/** không phải normative authority.
Product Knowledge	Giải thích WHY, broader WHAT, mục đích module, quan hệ, scope và non-goals.	Không tự chứng minh code hiện tại, deployment hoặc readiness.
Normative main specs	Sau approved sync, định nghĩa yêu cầu hành vi chính xác trong durable boundaries đã chấp nhận.	Không thay thế ADR, security/runtime/data authority, code hoặc bằng chứng production.
Code/tests	Chứng minh repository Implemented State và mức độ test coverage.	Không tự tạo Product approval hoặc chứng minh bản nào đang chạy production.
QA evidence	Chứng minh hành vi người dùng/runtime, responsive, accessibility và visual khi áp dụng.	Screenshot không định nghĩa business rule, permission, schema, owner hoặc lifecycle.
Human reviewer / current user	Ra quyết định tại đúng gate trên đúng artifact/hashes; cấp authorization rõ ràng khi cần.	Một approval không tự mở rộng sang gate, change, path hoặc phiên bản artifact khác.

Các lớp này bổ sung cho nhau. Khi chúng mâu thuẫn, dùng `AUTHORITY_MODEL.md` để phân loại câu hỏi, ghi CONFLICT và NEEDS REVIEW, rồi dừng ở authority phù hợp; không âm thầm chọn lớp thuận tiện nhất.

one authority layer does not silently replace another.

3. Tổng quan end-to-end

IDEA

→ Discovery/Shaping [conditional]

→ Proposal

→ Analysis

→ Gate 1

→ Specs

→ Gate 2

→ Design [when applicable]

→ Sensitive Design Gate [conditional]

→ Tasks + Implementation Plan

→ Apply

→ Verify

→ QA

→ Gate 3

→ Approval

→ Sync

→ Validate

→ Archive

→ Knowledge Consolidation

→ Done

Discovery/Shaping, Sensitive Design Gate, Gate 2 và normative promotion có điều kiện; các nhánh được giải thích bên dưới. Song song về mặt quản trị, nhưng chỉ bắt đầu sau khi có authorization phù hợp:

RELEASE / DEPLOY / POST-DEPLOY VERIFY /
MONITOR / ROLLBACK
= separate operational lane

Design conditionality là chính sách controlled-adapter của YUTA, không phải native conditional dependency của OpenSpec: raw graph vẫn phụ thuộc Design. Khi không applicable, ghi rõ lý do và authority/evidence trong tasks.md, giữ bằng chứng qua resume/review/finalization và không tạo placeholder design.md. Chỉ đi tiếp khi các prerequisite/gate khác đã thỏa mãn; raw planning có thể vẫn báo incomplete. Sensitive Design Gate được đánh giá độc lập theo tiêu chí nhạy cảm, không tự phát sinh chỉ vì có Design. skip_specs: true không tự bỏ Design.

4. Routing cross-module trước một change

Trước mọi feature/change mới, Page Chat thực hiện CROSS-MODULE IMPACT CHECK theo YUTA_PAGE_CHAT_OPERATING_PROMPT_V3.md và chọn đúng một kết quả:

Kết quả	Cách xử lý
PAGE_LOCAL	Ở lại owning Page Chat. Xác nhận capability, lý do page-local, nhu cầu Discovery/Shaping và OpenSpec readiness.
CROSS_MODULE	Không tạo/tiếp tục local change. Lập handoff theo Control Tower Handoff Template v3.
UNCERTAIN	Xử lý như CROSS_MODULE cho đến khi ownership và scope được làm rõ.

Một thay đổi cần Control Tower khi nó đọc/ghi data của nhiều owner, thay đổi shared permission/security/tenancy, đi qua nhiều runtime, cần provider/legal/ privacy review, chạm durable boundary, hoặc cần contract và QA phối hợp giữa nhiều module. Control Tower dùng YUTA_CONTROL_TOWER_OPERATING_PROMPT_V3.md để chọn một cross-module change, parent coordination với nhiều bounded changes, hoặc trả lại Page Chat nếu thực tế là page-local. Phân loại PAGE_LOCAL không phải miễn trừ kiểm tra sau đó. OpenSpec analysis là lớp bảo vệ thứ hai: nếu Codex phát hiện cross-module ownership, CONFLICT hoặc requirement-level NEEDS REVIEW, workflow dừng và chuyển coordination sang Control Tower.

5. Hướng dẫn vận hành từng bước

01-06 / 18 steps

Step / Main question

Discovery/Shaping / Request đã đủ rõ và đúng owner chưa?

Key work	Input / Output / Stop condition / next gate
Đọc authority và implementation read-only; làm rõ scope, owner, runtime/data/provider/cross-module uncertainty.	Input: Idea/request và current repository knowledge. Output: Câu hỏi cần quyết định, bounded request, hoặc kết luận không tạo change. Stop condition / next gate: Conditional; dừng nếu chưa thể bound an toàn, nếu rõ thì Proposal.

Proposal / Tại sao cần change và capability nào thay đổi?

Key work	Input / Output / Stop condition / next gate
Ghi Why, What Changes, Capabilities, Impact; xác định behavior-changing hay no-spec candidate.	Input: Bounded request. Output: proposal.md. Stop condition / next gate: Sang Analysis; chưa phải approval.

Analysis / Authority và evidence hiện tại có cho phép viết requirements không?

Key work	Input / Output / Stop condition / next gate
Phân loại Product Intent, Implemented State, runtime/data/security, lifecycle và unknowns.	Input: Proposal, Product Knowledge, authority/registry, code/tests khi cần. Output: analysis.md với một kết luận hợp lệ. Stop condition / next gate: Gate 1; blocker thì BLOCKED_NEEDS_REVIEW.

Gate 1 / Scope và authority có đúng để đi tiếp không?

Key work	Input / Output / Stop condition / next gate
Review exact Proposal + Analysis, conflicts, unknowns và hashes.	Input: proposal.md, analysis.md. Output: docs/reviews/<change>/01-analysis-review.md. Stop condition / next gate: Dừng chờ human review. Approval mở Specs hoặc no-spec branch.

Specs / Hệ thống phải có hành vi quan sát được nào?

Key work	Input / Output / Stop condition / next gate
Viết delta requirements/scenarios, không đưa implementation plan vào spec.	Input: Approved Gate 1 và artifact instructions. Output: specs/<capability>/spec.md deltas. Stop condition / next gate: Strict validation rồi Gate 2.

Gate 2 / Requirements có đúng, đủ và trong approved boundaries không?

Key work	Input / Output / Stop condition / next gate
Review exact deltas, scenarios, edge cases, assumptions, validation và hashes.	Input: Delta specs + Gate 1 evidence. Output: docs/reviews/<change>/02-specs-review.md. Stop condition / next gate: Dừng chờ human review; bỏ qua chỉ trên approved skip_specs: true.

5. Hướng dẫn vận hành từng bước

07-12 / 18 steps

Step / Main question

Design / Cách hiện thực nào giữ đúng requirements và boundaries?

Key work	Input / Output / Stop condition / next gate
Ghi decisions, alternatives, risks, migration/rollback và safe open questions khi design cần thiết.	Input: Approved requirements hoặc approved no-spec path. Output: design.md khi applicable. Stop condition / next gate: Sensitive classification; unresolved decision quay lại gate sở hữu nó.

Sensitive Design Gate / Thiết kế nhạy cảm có được authority phù hợp chấp nhận không?

Key work	Input / Output / Stop condition / next gate
Review security/auth, runtime/data owner, migration, payment/fiscal, Personnel/legal/privacy, provider, POS transaction, irreversible hoặc cross-module durable-boundary impact.	Input: Exact Design + earlier approvals. Output: docs/reviews/<change>/02b-design-review.md. Stop condition / next gate: Conditional; dừng trước Tasks/Apply cho đến explicit approval.

Tasks / Làm việc gì, theo dependency nào, và chứng minh completion ra sao?

Key work	Input / Output / Stop condition / next gate
Chọn phase cần thiết; thêm checkbox outcomes và embedded TECHNICAL IMPLEMENTATION CONTRACT.	Input: Approved Specs/Design hoặc no-spec path. Output: tasks.md + implementation plan; UI_AFFECTING/BROWSER_QA_REQUIRED classification. Stop condition / next gate: Dừng nếu owner/boundary/constraint chưa giải quyết; nếu sẵn sàng thì Apply.

Apply / Implementation có tạo đúng approved outcome không?

Key work	Input / Output / Stop condition / next gate
Thực hiện code/DB/UI/tests theo phases và contracts; đánh dấu task chỉ khi outcome + evidence tồn tại.	Input: Approved planning artifacts, clean scoped baseline. Output: Scoped implementation + task evidence. Stop condition / next gate: Durable/Product discovery quay lại gate; technical defect trong scope có thể sửa rồi tiếp tục.

Verify / Repository implementation có khớp Specs/Design và technical authorities không?

Key work	Input / Output / Stop condition / next gate
Mapping requirements, tests, typecheck/build, architecture/security, migration/schema, diff và Compliance Matrix.	Input: Implementation + approved planning. Output: TECHNICAL IMPLEMENTATION COMPLIANCE và VERIFY result. Stop condition / next gate: Không PASS thì sửa trong approved behavior hoặc quay lại gate phù hợp.

QA / Hành vi có hoạt động đúng trong user/runtime context không?

Key work	Input / Output / Stop condition / next gate
Chạy Browser QA hoặc non-browser QA phù hợp; lưu report/evidence.	Input: Verified implementation và realistic environment/data. Output: PASS, FAIL, BLOCKED_BY_ENVIRONMENT, hoặc NOT_APPLICABLE. Stop condition / next gate: Required QA chưa PASS thì chưa được tạo ready Gate 3.

5. Hướng dẫn vận hành từng bước

13-18 / 18 steps

Step / Main question

Gate 3 / Exact implementation đã đủ an toàn để final approval chưa?

Key work	Input / Output / Stop condition / next gate
Independent review planning hashes, attributed diff, Technical Compliance, VERIFY, QA, deviations và lifecycle truth.	Input: Toàn bộ current evidence. Output: docs/reviews/<change>/03-final-review.md. Stop condition / next gate: Dừng chờ explicit Gate 3 approval và explicit sync/archive authorization.

Sync / Approved deltas nào được mechanical promotion?

Key work	Input / Output / Stop condition / next gate
Recheck hashes; sync đúng existingOutputPaths; review exact main-spec diff.	Input: Approved Gate 3 + explicit authorization. Output: Updated normative main specs, hoặc không có promotion trên no-spec path. Stop condition / next gate: Partial/unexpected sync dừng và rollback có ghi nhận; thành công thì Validate.

Validate / Main-spec result có hợp lệ và đúng approved delta không?

Key work	Input / Output / Stop condition / next gate
Strict main-spec validation và diff review.	Input: Synced main specs. Output: Validated main-spec state. Stop condition / next gate: Failure: không archive như thành công; success: Archive.

Archive / Change history có thể đóng an toàn chưa?

Key work	Input / Output / Stop condition / next gate
Reconfirm task/artifact completion và sync state; archive đồng bộ.	Input: Validated sync hoặc valid no-spec finalization. Output: Recorded archive. Stop condition / next gate: Archive thành công mới sang Knowledge Consolidation.

Knowledge Consolidation / Current knowledge có cần cập nhật từ completed evidence không?

Key work	Input / Output / Stop condition / next gate
Scan đúng các knowledge authorities có thể bị ảnh hưởng; không tự promote status.	Input: Archived completed change. Output: NO_UPDATE_REQUIRED hoặc UPDATE_REQUIRED + review packet. Stop condition / next gate: Update required thì dừng chờ Knowledge Review; no update thì Done.

Done / Repository workflow đã đóng trung thực chưa?

Key work	Input / Output / Stop condition / next gate
Ghi workflow outcome và RELEASE_FOLLOW_UP.	Input: Completed consolidation path. Output: Workflow status: DONE. Stop condition / next gate: Chỉ chuyển sang release/deploy lane nếu được yêu cầu và authorized.

6. Analysis gate

YUTA thêm analysis để ngăn việc một request đi thẳng từ ý tưởng sang requirements bằng assumption. Analysis phải kiểm tra:

- Product Knowledge cho broader intent và scope;
- Authority Model để chọn nguồn kiểm soát theo loại câu hỏi;
- Module Registry và Lifecycle Status Model cho owner, relationship và trạng thái bounded hiện tại;
- code/tests/contracts/executable schemas chỉ để xác minh Implemented State;
- runtime, data ownership, security/authorization và production evidence theo đúng authority;
- mọi CONFLICT và NEEDS REVIEW còn tồn tại.

Kết luận Analysis hợp lệ **chính xác** là:

```
READY_FOR_SPECS
BLOCKED_NEEDS_REVIEW
NO_SPEC_BEHAVIOR_CHANGE
```

CONFLICT là một finding (phát hiện), không phải Analysis conclusion. Nếu conflict ảnh hưởng requirement readiness, conclusion phải là BLOCKED_NEEDS_REVIEW; approval chung chung không giải quyết được blocker.

8. Tasks, phased implementation và Technical Implementation Contract

Các phase sau là nhãn lập kế hoạch tùy chọn, không phải chuỗi stage bắt buộc:

- Foundation / Data
- Service / Domain
- UI / Components
- Interaction / States
- Integration / Regression

Chỉ dùng phase mà change thật sự cần và sắp theo dependency. Trong mỗi phase đã chọn, tasks.md nhúng một TECHNICAL IMPLEMENTATION CONTRACT ghi:

- boundary bị ảnh hưởng và canonical owner;
- root/nearest scoped AGENTS.md và các repository authority áp dụng;
- constraints thật sự liên quan;
- intended files/packages;
- required checks;
- completion evidence.

Đây không phải OpenSpec artifact mới. Code, DB, UI và test thật được thay đổi ở APPLY, không phải khi viết contract. Một task chỉ complete khi implementation outcome và compliance evidence cùng tồn tại. Nếu cần permission, API, contract, owner hoặc durable boundary mới, quay lại authority/gate phù hợp thay vì nói Specs/Design để khớp code.

7. Review gates và integrity

Gate	Reviewer đang quyết định gì? / Review packet
Gate 1 — Product / Authority Review	Proposal/Analysis có đúng scope, owner, authority và đủ rõ để viết Specs hoặc đi no-spec không? Review packet: 01-analysis-review.md
Gate 2 — Requirements Review	Exact requirements/scenarios có đúng approved behavior, không chứa hidden assumption và strict-valid không? Review packet: 02-specs-review.md
Sensitive Design Gate	Thiết kế nhạy cảm có giữ đúng durable, security, data/runtime, legal/provider và rollback boundaries không? Review packet: 02b-design-review.md
Gate 3 — Final Independent Review	Exact implementation/evidence có đủ để approve finalization và cho phép sync/archive không? Review packet: 03-final-review.md

PASS, test xanh, file tồn tại, commit/PR approval hoặc recommendation của Codex không thay thế human decision. Mỗi approval bị ràng buộc bởi exact path set và SHA-256 của bytes đã review. Khi thêm, xóa, đổi tên hoặc thay bytes của artifact/evidence, packet liên quan trở thành INVALIDATED_BY_ARTIFACT_CHANGE và phải được review lại.

Các trạng thái review packet là AWAITING_HUMAN_REVIEW, APPROVED, CHANGES_REQUESTED, và INVALIDATED_BY_ARTIFACT_CHANGE. Approval phải đến từ current user, ghi rõ change/gate đang duyệt; Gate 3 còn cần authorization rõ ràng cho sync và archive.

9. VERIFY khác QA

VERIFY — technical correctness

VERIFY trả lời “implementation trong repository có khớp approved Specs/Design và technical authorities không?”. Evidence có thể gồm:

- Specs/Design → code/test mapping;
- targeted và broader tests;
- typecheck, build và strict OpenSpec validation;
- architecture, security, schema và migration checks;
- scoped diff và deviations;
- TECHNICAL COMPLIANCE MATRIX ánh xạ constraint → authority → implementation → check/evidence → PASS | FAIL.

QA — user/runtime correctness

QA trả lời “hành vi có hoạt động đúng trong ngữ cảnh người dùng/runtime không?”. Nó bao gồm browser interaction, responsive behavior, roles/states, loading/error/success/recovery, accessibility, visual behavior và screenshots khi applicable.

```
QA status:
PASS
FAIL
BLOCKED_BY_ENVIRONMENT
NOT_APPLICABLE
```

Thay đổi UI-visible, interaction, responsive, role/edit/read-only state hoặc loading/error/success presentation bắt buộc Browser QA trên real/local route với boundary xác thực/persistence thật.

Backend/data-only không cần Browser QA vô nghĩa; correctness của schema, migration, repository, tenancy, authorization và integration thuộc VERIFY.

Chi tiết evidence dùng YUTA_QA_PROTOCOL.md.

11. Normative specs và sync

```
approval permits sync
sync performs mechanical promotion
```

openspec/changes/** luôn là proposed/in-progress và non-normative. Nội dung main spec chỉ trở thành normative cho precise observable behavior sau khi exact delta qua accountable approval gate, current user cấp explicit sync authorization, sync thành công, và resulting main specs qua validation + diff review.

Sync không tạo Product approval, không thay accepted durable boundaries và không tự cập nhật lifecycle. Main specs cũng không chứng minh implementation, deployment, database shape hoặc Production Readiness. Xem OPENSPEC_YUTA_NORMATIVITY_POLICY_REVIEW.md cho conflict, rollback và modification policy chi tiết.

10. Gate 3 readiness

Gate 3 chỉ ready khi:

```
TECHNICAL IMPLEMENTATION COMPLIANCE = PASS
VERIFY = PASS
required QA = PASS
```

Với non-UI change, QA: NOT_APPLICABLE chỉ hợp lệ khi thật sự không có user/ runtime QA dimension. FAIL, BLOCKED_BY_ENVIRONMENT, thiếu role/state, responsive coverage hoặc evidence bắt buộc đều ngăn ready Gate 3.

UI-affecting change lưu evidence tại:

```
docs/reviews/<change-name>/qa/
├── QA_REPORT.md
├── screenshot-manifest.md
└── *.png
```

Mỗi screenshot phải đến từ actual Browser QA và manifest ghi path, viewport, role/state, scenario cùng lowercase SHA-256 của exact file bytes. Viewport của page pack được ưu tiên; nếu không có, tối thiểu desktop 1366x768 và mobile 390x844, thêm intermediate/tablet khi layout hoặc target yêu cầu.

12. \$yuta-run-change

Đây là workflow hằng ngày để bắt đầu một bounded change, resume một change hoặc adopt một in-flight change. Nó luôn tìm gate sớm nhất còn thiếu, chưa duyệt, changes-requested hoặc invalidated; artifacts về sau không được bypass gate trước. Nó tự làm việc giữa các gate, dừng ở human review kế tiếp, và **không bao giờ sync hoặc archive**.

Ví dụ bắt đầu:

```
$yuta-run-change <mô-tả bounded change>
```

Ví dụ resume sau một approval cụ thể:

```
$yuta-run-change <change-name>
Analysis review approved. Continue.
```

```
$yuta-run-change <change-name>
Specs review approved. Continue.
```

Với in-flight change, workflow giữ nguyên bytes hiện có, tạo packet cho gate sớm nhất cần review và không tự “sửa cho đẹp” artifact đã tồn tại.

13. \$yuta-finish-change

Hai mode có precondition và integrity scope riêng, không được trộn.

Branch A — Active finalization

```
Gate 3 approval + explicit sync/archive
authorization
→ integrity recheck
→ sync (hoặc no normative promotion trên valid
no-spec path)
→ validate
→ archive
→ knowledge scan
```

Lệnh/resume intent điển hình:

```
$yuta-finish-change <change-name>
Final review approved. I authorize spec sync
and archive.
```

Workflow recheck planning artifacts, attributed implementation diff, VERIFY, Technical Compliance, earlier gates và applicable QA hashes trước khi ghi approval. Drift dừng finalization và trả change về \$yuta-run-change.

Branch B — Archived Knowledge Review resume

```
Gate 3 already approved + archive completed
→ approve exact knowledge diff
→ recheck target path/hashes + proposed-diff
hash
→ apply exact approved documentation update
→ validate docs
→ DONE
```

Ví dụ intent:

```
$yuta-finish-change <change-name>
Knowledge consolidation review approved. Apply
and close.
```

Branch B không tái tạo active change, không yêu cầu Gate 3 approval mới, không rerun sync/archive và không dùng lại authorization cũ để cấp quyền sửa docs.

14. Knowledge Consolidation

Archive chưa phải bước cuối. Sau archive, scan có giới hạn:

- Page Product Knowledge và page-pack/as-built;
- Module Product Knowledge;
- docs/PRODUCT_KNOWLEDGE.md routing;
- Module Registry và lifecycle/current-state;
- ADR/durable decisions;
- CURRENT_STATE.md khi broad summary thay đổi thực chất;
- NEEDS REVIEW thật sự được completed change giải quyết;
- limitation hoặc future work mới phát hiện.

```
NO_UPDATE_REQUIRED
→ record reason + sources inspected
→ DONE

UPDATE_REQUIRED
→ docs/reviews/<change>/04-knowledge-consolidation-review.md
→ human review
→ recheck exact hashes
→ apply approved docs diff
→ validate
→ DONE
```

Knowledge Consolidation không tự approve Product Decision, đổi durable boundary/owner/permission/API, promote lifecycle/readiness, rewrite normative specs hoặc giải quyết NEEDS REVIEW bằng assumption. Chi tiết nằm trong

YUTA_KNOWLEDGE_CONSOLIDATION_PROTOCOL.md.

17. Ví dụ hằng ngày

Example A — small page-local UI change

Page Chat xác nhận PAGE_LOCAL; nếu scope đã rõ thì bỏ Discovery/Shaping. Change đi qua Proposal → Analysis → Gate 1 → Specs → Gate 2; Design chỉ tạo khi applicable và không có Sensitive Design Gate nếu không chạm boundary nhạy cảm. Tasks chọn UI / Components, Interaction / States và regression cần thiết. Sau Apply, VERIFY chứng minh mapping/code/tests; Browser QA chạy real route ở viewport theo page pack, kiểm tra relevant state/accessibility và lưu screenshot hashes. Gate 3 chỉ ready khi Technical Compliance, VERIFY và QA đều PASS.

15. No-spec path

skip_specs: true chỉ dùng khi thật sự không có spec-level behavior change, ví dụ pure refactor, tooling hoặc docs. Không tạo requirement giả để vượt validation.

```
Gate 1 approved
→ no Gate 2
→ no normative spec promotion
→ continue applicable
Design/Tasks/Apply/Verify/QA
→ Gate 3
→ Archive
→ Knowledge Consolidation
```

Gate 3 vẫn cần human approval và explicit finalization authorization; mọi Technical Compliance, VERIFY và applicable QA rule vẫn áp dụng. Nếu delta specs tồn tại trên một claimed no-spec path, đó là conflict và workflow phải dừng.

16. Release / Deploy lane

Sau repository DONE, ghi đúng một giá trị:

```
RELEASE_FOLLOW_UP: NOT_REQUIRED | REQUIRED
| UNKNOWN
```

Nếu required, xác định runtime/environment, deployment authorization, migration hoặc provider/readiness evidence, post-deploy verification, monitoring và rollback. Đây là operational work riêng; Workflow v3 không tự deploy và không dùng QA repository để tuyên bố production enabled.

17. Ví dụ hằng ngày

Example B — backend/data-only change

Sau các Product/requirements/design gate applicable, Tasks có thể dùng Foundation / Data, Service / Domain và Integration / Regression. VERIFY phải chứng minh schema/migration, repository behavior, tenant isolation, authorization, integration và rollback concerns. Không ép Browser QA nếu không có UI; QA: NOT_APPLICABLE chỉ dùng khi cũng không có distinct runtime/user QA dimension. Gate 3 vẫn review exact technical evidence và diff.

Example C — cross-module feature

Page Chat thực hiện impact check và trả CROSS_MODULE hoặc UNCERTAIN, lập handoff rồi dùng local change. Control Tower map owning/consumer capabilities, data owners, authority, shared contract và quyết định một coordinated change hay nhiều bounded changes. Chỉ sau khi Product/durable blockers được giải quyết, Control Tower mới cấp bounded \$yuta-run-change request và routing cho các gate/QA liên quan.

18. Quick reference

CORE

```
IDEA → [Discovery] → Proposal → Analysis →
Gate 1
→ Specs → Gate 2 → [Design Gate] → Tasks →
Apply → Verify → QA
→ Gate 3 → Approval → Sync → Validate →
Archive
→ Knowledge Consolidation → DONE
```

PACKETS

```
01-analysis-review.md
02-specs-review.md
02b-design-review.md conditional
qa/QA_REPORT.md required for UI-affecting
qa/screenshot-manifest.md required for
UI-affecting
03-final-review.md
04-knowledge-consolidation-review.md
conditional after archive
```

COMMANDS

```
$yuta-run-change = start/resume/adopt; stop at
next human gate
$yuta-finish-change = authorized finalization
OR archived knowledge resume
```

ESCALATION

```
PAGE_LOCAL → owning Page Chat
CROSS_MODULE / UNCERTAIN → YUTA Control Tower
Codex finding CONFLICT / requirement-level
NEEDS REVIEW → stop and route
```

QA

```
UI-affecting → Browser QA +
responsive/state/accessibility + hashed
screenshots
Backend/data-only → technical correctness in
VERIFY; no meaningless Browser QA
```

KNOWLEDGE

```
Archive → scan → NO_UPDATE_REQUIRED or
reviewed exact knowledge diff → DONE
```

Các bất đẳng thức phải giữ nguyên:

```
APPROVED != IMPLEMENTED
IMPLEMENTED != PRODUCTION_ENABLED
VERIFY != QA
QA PASS != PRODUCTION_READINESS
ARCHIVE != KNOWLEDGE_CURRENT
SYNC != PRODUCT_APPROVAL
WORKFLOW_PROGRESS != LIFECYCLE_PROMOTION
```